Ders 5 Genişletilmiş Notu: Özyinelemeli Algoritmalar ve Yineleme Denklemleriyle Analiz

Ders 5 Genişletilmiş Notu: Özyinelemeli Algoritmalar ve Yineleme Denklemleriyle Analiz

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Özyineleme Nedir?
- 2. Özyinelemeli Yapının Üç Şartı
- 3. Hatalı Bir Örnek: Kendini Çağıran `main`
- 4. Üs Alma Örneği
- 5. Özyinelemenin Artıları ve Eksileri
- 6. Gizemli Fonksiyon: `mystery`
- 7. Hanoi Kuleleri
- 8. Onluk Sayıyı İkilik Tabanda Yazdırma
- 9. Özyinelemeli Algoritmalarda Büyük O: Yineleme Denklemi
- 10. Sık Görülen Yineleme Denklemleri
- 11. Özyinelemeli İkili Arama ve Yineleme Denklemi
- 12. Yineleme Denklemlerini Çözme Yöntemleri ve İterasyon
- 13. Özyinelemeli Fibonacci
- 14. Özyineleme Ağacı: `fib(6)`
- 15. Daha İyi Çözüm: Döngüyle Fibonacci

1. Özyineleme Nedir?

Hoca bu haftanın konusunu özyinelemeli (recursive) algoritmalar olarak açtı. Öğrencilerin özyinelemeyi önceki derslerinden zaten tanıdığını, ama bu derste konuya özellikle algoritma tasarımı açısından bakılacağını söyledi. Dersin ikinci hedefi de özyinelemeli bir algoritmanın karmaşıklığının temelde nasıl analiz edileceğini görmekti.

Teknik tanım şöyledir: Çözülmesi gereken bir problemi aynı türden daha küçük alt problemlere indirmek ve bütün probleme uyguladığımız çözümün aynısını daha küçük problem örneği üzerinde tekrar işletmek. Bu esasa dayanan çözüm yaklaşımına özyinelemeli algoritma tasarımı denir.

Hocaya göre özyinelemeli yapılar, programlama dilleri açısından çok güçlü araçlardır. Kodlamayı çok kolaylaştırır ve kısaltırlar. Ama bunun bir bedeli vardır: Algoritmanın karmaşıklığını, yani çalışma zamanı ihtiyacını genellikle artırırlar. Bu yüzden en iyi çözümü her zaman özyinelemeyle elde edebildiğimizi söyleyemeyiz. Öte yandan literatürde bilinen bazı önemli problemleri çözmek için özyineleme gereklidir; bazıları için belki de tek yoldur.

2. Özyinelemeli Yapının Üç Şartı

Bir özyinelemeli algoritma kurabilmek için üç şart gerekir.

- **Tüm geçerli girdileri karşılayan bir durum olmalıdır.** Hoca "tüm girdileri" kelimesinin altını çizdi. Girdi uzayında tanımlı her geçerli değerin bir karşılık bulacağı bir akış kurulmalıdır. "Bazı girdiler için çalışmaz ya da tanımsızdır" gibi bir durum olamaz.
- **Hiç özyineleme içermeyen bir temel durum (base case) olmalıdır.** Özyineleme, kendini tekrar tekrar daha küçük problemler üzerinde çağırmak demektir. Problemin boyutunu, örneğin birer birer azaltırız. Bir yerde kendimizi çağırmayı kesmemiz gerekir ki özyineleme dursun. Ardından oluşan sonuçları üst üste koyarak ilk çağrıya geri dönebilelim.
- **Her özyinelemeli çağrı problem uzayındaki daha küçük ya da daha basit bir örneğe yapılmalı ve temel duruma doğru ilerlemelidir.** Bu, sona erdirilebilme şartıdır. Küçülen örnek miktarı her koşulda bir noktada temel duruma varmalıdır. Bunu sağlamazsak sonsuza kadar kendimizi çağırırız ve sonuçları toplama noktasına hiç varamayız.

Bu üç şart sağlanıyorsa özyinelemeli bir yapı kurulmuş demektir.

3. Hatalı Bir Örnek: Kendini Çağıran `main`

Hoca üç satırlık bir kod gösterdi:

```
int main() {
    main();
    return 0;
}
```

`main` fonksiyonu kendi içinde yine `main`'i çağırıyor ve 0 döndürüyor. Hoca bunun özyinelemeli bir algoritma olup olmadığını üç şart üzerinden sınadı:

- **Birinci şart:** Geçerli girdi kümesi boştur. Fonksiyon parametre almadığı için boş kümeyi karşılıyor sayılabilir.
- **İkinci şart:** `main` içine girince yine `main` çağrılır, onun içinde bir daha, sonra bir daha. Kendini çağırma sonsuza kadar sürer. Temel durum tanımlanmamıştır.
- **Üçüncü şart:** Hiçbir küçülme yoktur. Çağrı, tanımlı bir değere doğru ilerlemez.

Sonuç olarak bu kod, özyinelemeli bir algoritmanın gerçeklemesi sayılmaz. Doğru bir özyineleme tanımı değildir.

> **Ek bilgi (derste anlatılmadı):** Bu kod `gcc -Wall` ile derlendiğinde derleyici "infinite recursion detected" (sonsuz özyineleme) uyarısı verir. Kod derlenir, ama çalıştırılırsa çağrı yığını dolana kadar kendini çağırır ve program çöker.

4. Üs Alma Örneği

Hoca doğru bir özyinelemeli algoritma için üs alma (power) problemini seçti.

Herhangi bir sayının sıfırıncı kuvveti 1'dir: $x^0 = 1$. Burada kendini tekrar çağırma yoktur. Bu, ikinci şartın istediği temel durumdur.

Genel durum için: Bir sayının n. kuvvetini almak, o sayıyı kendisinin $(n-1)$. kuvvetiyle çarpmak demektir: $x^n = x \cdot x^{n-1}$.

Hocanın örneği 5^3 idi. Adım adım:

- $5^3 = 5 \cdot 5^2$. Problem 3'ten 2'ye indi. Daha basit bir örneğe doğru ilerledik.
- $5^2 = 5 \cdot 5^1$. Üs 2'den 1'e indi.
- $5^1 = 5 \cdot 5^0$. Üs 1'den 0'a indi.
- $5^0 = 1$. Bu temel durumdur; kendini çağırmaya gerek yoktur.

Sonra geri dönüş başlar:

- $5^1 = 5 \cdot 1 = 5$.
- $5^2 = 5 \cdot 5 = 25$. Hoca, 5^2 yerine 25 yazabilmemizi önceki iki çağrının sonuçlarının birleşmesine bağladı.
- $5^3 = 5 \cdot 25 = 125$.

Kendini giderek küçülen örneklerle çağıran ve temel duruma inen bu tür yapılara özyinelemeli fonksiyon denir.

4.1 Kod ve Çağrıların İzlenmesi

```
int power(int x, int exp) {
    if (exp == 0)
        return 1;                    /* temel durum */
    else
        return x * power(x, exp - 1);
}

printf("%d", power(5, 3));
```

Hoca akışı da kısaca çizdi: Fonksiyon x ve üssü (exp) parametre olarak alır. Tek bir `if` vardır. Üs sıfıra eşitse 1 döndürülür. Değilse fonksiyon kendini x ve bir azaltılmış üsle çağırır.

`printf("%d", power(5, 3))` satırı çalışınca çağrılar şöyle yapılır:

- 1. çağrı: `power(5, 3)`. Üs 0 değil; `5 * power(5, 2)` hesaplanmalı.
- 2. çağrı: `power(5, 2)`. Üs 0 değil; `5 * power(5, 1)` hesaplanmalı.
- 3. çağrı: `power(5, 1)`. Üs 0 değil; `5 * power(5, 0)` hesaplanmalı.
- 4. çağrı: `power(5, 0)`. $x = 5$, üs $= 0$. Şart doğru olduğu için `return 1` çalışır.

Geri dönüşte:

Çağrı	Çalışan kol	Dönen değer
`power(5, 0)`	temel durum, `return 1`	1
`power(5, 1)`	`else`, `5 * power(5, 0)` = $5 \cdot 1$	5
`power(5, 2)`	`else`, `5 * power(5, 1)` = $5 \cdot 5$	25
`power(5, 3)`	`else`, `5 * power(5, 2)` = $5 \cdot 25$	125

Son değer 125, `power(5, 3)`'ü çağıran `printf`'e döner ve ekrana 125 yazılır.

Kendini çağıran, ama giderek azalan bir uzayda çağıran, bir noktada mutlaka temel duruma inen ve sonra çağrıları teker teker geri dönerek sonucu ileten yapı kurulduğunda özyinelemeli bir algoritma tasarlanmış olur.

5. Özyinelemenin Artıları ve Eksileri

Bu noktada bir öğrenci sordu.

Soru: Özyinelemeli fonksiyonlar bize ne kazandırıyor? Hızdan ya da bellekten bir kazancımız var mı, yoksa yalnız kodlama kolaylığı mı sağlıyor?

Cevap: Hoca, kazancın kodlama kolaylığı ve akışın daha basit ifade edilmesi olduğunu söyledi. Buna karşılık iki götürüsü vardır:

- **Zaman:** Her zaman değil ama genellikle, örneğin bir döngü içinde düz bir şekilde çözebileceğimiz bir problemden daha uzun sürede çalışırız. Algoritmanın zaman karmaşıklığı artar.
- **Bellek:** Daha fazla bellek ihtiyacı doğar.

Hoca bellek ihtiyacının sebebini `power(5, 3)` üzerinden anlattı. `power(5, 3)` içinden `power(5, 2)` çağrılır. Bu çağrıya nereden gelindiği bir yerde tutulmalıdır. `power(5, 2)` içinden `power(5, 1)` çağrılır; ona `power(5, 2)`'den gelindiği de unutulmamalıdır. `power(5, 1)` içinden de `power(5, 0)` çağrılır. Temel durum 1 üretir. Bu 1, onu çağıran `power(5, 1)`'e dönmelidir; o da $5 \cdot 1 = 5$ üretir ve kendini çağırana döner. Sonra 25 oluşur ve o da kendini çağırana döner.

Yani dört fonksiyon çağrısının hangi sırayla yapıldığı bir yerde tutulmalıdır ki çağrılar ters sırada işlenebilsin: En son çağrılan en önce işlenir; en önce çağrılan en son işlenir. Fonksiyonun nerede kaldığı, kendini nerede ve hangi parametrelerle çağırdığı gibi ek bilgi derleyici tarafından bir yerde tutulmalıdır. Buna ek yük (overhead) denir ve fazladan bellek gerektirir.

Hoca artı ve eksileri şöyle topladı:

- **Eksi 1:** Döngüyle doğrusal çözülebilecek problemlere göre genellikle daha uzun süren çözümler üretir; algoritmik karmaşıklık daha yüksektir.
- **Eksi 2:** Fonksiyon çağrılarını hatırlamak gerektiği için daha fazla bellek gerekir.
- **Artı:** Kodun yazma yükü çok azdır. `power` için tek bir fonksiyon yazdık. Temel durumu tanımladık ve geri dönüşteki toplamayı `x * power(x, exp - 1)` ile sağladık.

İkinci bir öğrenci sorusu bellek sınırıyla ilgiliydi.

Soru: Örneğin $1\,000\,000^{1\,000\,000}$ gibi bir sayıyı hesaplamak istersek, stack overflow olmaması için derleyici bu yeri dinamik olarak mı ayarlıyor?

Cevap: Hoca, yerin dinamik ayarlandığını ama belleğin fiziksel sınırları olduğunu söyledi. Derleyicinin adresleyebildiği bir üst sınır da vardır. $10\,000^{10\,000}$ ya da $1\,000\,000^{1\,000\,000}$ gibi sayıları bu yolla hesaplamaya çalışırsanız büyük olasılıkla program çöker. Bu yüzden özyineleme kodlamayı kolaylaştırır ama her zaman iyi bir çözüm değildir. Hoca, gelecek yıl algoritma analizi dersinde bu tür problemlerin dinamik programlamayla nasıl çözülebildiğinin görüleceğini ekledi.

> **Ders sonrası not:** Bu hesapta `power` fonksiyonu kendini 10^6 derinlikte çağırır; tipik bir çağrı yığını bu derinlikte büyük olasılıkla taşar. Ayrıca sonuç zaten `int` ya da `long` türüne sığmaz. `power`'ın yineleme denklemi $T(n) = T(n-1) + c$'dir ve $O(n)$ sürede çalışır (bkz. Bölüm 10).

6. Gizemli Fonksiyon: `mystery`

Hoca, algoritmayı bir akış olarak da yazabileceğini ama slaytta kod üzerinden anlatmanın daha basit olduğunu söyleyip ikinci örneğe geçti:

```
int mystery(int n) {
    int a, b;
    if (n < 10)
        return n;        /* temel durum */
    a = n / 10;
    b = n % 10;
    return mystery(a + b);
}
```

Fonksiyon bir n tam sayısı alır. $n < 10$ ise kendini çağırmadan n'yi döndürür; bu temel durumdur. $n \geq 10$ ise $a = n$ div 10 ve $b = n$ mod 10 hesaplanır ve fonksiyon kendini $a + b$ ile, yine tek bir tam sayıyla çağırır.

Hoca `mystery(648)` için sonucu sordu ve birlikte izlediler:

- $n = 648$. $648 < 10$ değil; temel durumda değiliz. $a = 64$, $b = 8$. `mystery(64 + 8)`, yani `mystery(72)` çağrılır.
- $n = 72$. $72 < 10$ değil. $a = 7$, $b = 2$. `mystery(9)` çağrılır.
- $n = 9$. $9 < 10$; özyineleme biter. 9 döndürülür.

9, `mystery(72)`'ye döner; o da 9'u hiç değiştirmeden kendini çağırana, `mystery(648)`'e döndürür. Sonuç 9'dur.

Hoca bu örneği üs almayla karşılaştırdı: Üs almada dönen değer x ile çarpılıyordu. Burada ise fonksiyon dönen değerle hiçbir şey yapmadan kendini çağırıyor.

Hoca ayrıca bir özyinelemeli fonksiyonun kendini yalnız bir kez çağırmak zorunda olmadığını vurguladı. Örneğin Fibonacci sayısı hesaplanırken kendinden bir önceki ve iki önceki değerler için iki çağrı yapılır. Özyineleme içinde birden fazla kez, farklı parametrelerle kendimizi çağırıp daha az örnek üzerinde çalıştırabiliriz.

> **Ek bilgi (derste anlatılmadı):** `mystery`, bir sayının basamakları tek basamak kalana kadar toplanınca elde edilen basamak kökünü (digital root) bulur. Örneğin $6 + 4 + 8 = 18$ ve $1 + 8 = 9$ olduğu için 648'in basamak kökü 9'dur. Fonksiyon bunu her adımda yalnız son basamağı ayırarak yapar: $64 + 8 = 72$, sonra $7 + 2 = 9$.

7. Hanoi Kuleleri

Hoca Hanoi Kuleleri (Tower of Hanoi) problemine geçmeden önce Hanoi'nin ne olduğunu sordu. Bir öğrenci aradı ve Vietnam'ın başkenti olduğunu buldu.

Problem, aslında bir çocuk oyunu, bir zeka oyunudur. Slaytta 5 diskli versiyon gösterildi; farklı disk sayılı versiyonlar da vardır, ama en çok oynanan bu 5 diskli olandır. A çubuğunda diskler en büyüğü altta, en küçüğü üstte olacak şekilde kule biçiminde dizilidir. Amaç bu kuleyi aynı yapıda başka bir çubuğa, örneğin C'ye taşımaktır.

Hoca önce kuralları bilmeyen birinin "çözümünü" canlandırdı: En üstteki kırmızıyı C'ye koy, turuncuyu onun üstüne, sonra sarıyı, yeşili, en son maviyi taşı ve hepsini geri diz. Ama bu yapılamaz, çünkü oyunun kuralları vardır:

- Her hamlede yalnız bir disk taşınabilir.
- Alttaki disk her zaman üstündekinden büyük olmalıdır; büyük disk küçüğün üstüne konamaz.

7.1 Adım Adım Denemek

Hoca öğrencilerle birlikte elle denedi. Kırmızı bir çubuğa, turuncu diğerine konursa sarıyı koyacak yer kalmaz: Ne kırmızının ne turuncunun üstüne konabilir. Bir öğrencinin önerisiyle kırmızı turuncunun üstüne alındı, sarı C'ye kondu. Öğrenci B'deki ikiliyi C'ye almayı önerince hoca, her adımda yalnız bir disk taşınabildiğini hatırlattı. Kırmızı bir yere, turuncu sarının üstüne, kırmızı da turuncunun üstüne alınarak devam edildi. Böylece adım adım, küçükler bir tarafa, büyükler diğer tarafa geçecek şekilde ilerlenir.

7.2 Özyinelemeli Fikir

Bir T anında üstteki dört disk bir çubukta toplanmış, en büyük (mavi) disk tek başına kalmış olur. Mavi disk hedef çubuğa taşınır. Hocanın vurguladığı nokta şudur: Bu aşamaya gelindiğinde elimizdeki problem artık 5 diskli değil, 4 diskli bir problemdir.

- 5 diskli problemi çözmek: 4 diski yedek çubuğa taşı, en büyük diski hedefe taşı, sonra 4 diskli problemi çöz.
- 4 diskli problemi çözmek: 3 diski yedeğe al, bir diski hedefe taşı, 3 diskli problemi çöz.
- 3 diskli problem: 2 diski yedeğe al, bir diski taşı.
- 2 diskli problem: Bir diski yedeğe al, birini taşı.
- 1 diskli problem temel durumdur: Yedeğe alacak bir şey kalmadığı için disk doğrudan hedef çubuğa (C) taşınır.

Hoca hedef çubuğun yanında, büyüklük sırası kuralı yüzünden B gibi bir yedek çubuğa mutlaka ihtiyaç olduğunu söyledi. Buna ilave mil, yedek mil ya da destek mili gibi adlar verilir. Birden fazla disk varsa bu yedek çubuk mutlaka kullanılmalıdır.

Hoca, amacın süreci daha az elemandan oluşan, düşünmesi değil uygulaması basit, körlemesine yapılabilecek ve hep aynı şeyi tekrar eden adımlara indirmek olduğunu söyledi.

Hoca algoritmayı derste çizmedi; zaman sınırı ve slaytta çizmenin zorluğu nedeniyle öğrencilerden internetteki kaynaklardan incelemelerini istedi. Algoritmada şunlar parametre olarak tutulmalıdır: hangi diskin taşındığı, hangi çubuktan hangi çubuğa taşındığı, hedefin neresi olduğu. Yani A, B, C çubuklarının adresleri sağlanmalıdır. (Standart çözüm "Örnek Sorular" bölümündedir.)

Derse burada ara verildi.

8. Onluk Sayıyı İkilik Tabanda Yazdırma

Aradan sonra hoca yeni bir problem verdi: Onluk tabanda (decimal) verilen bir n tam sayısını ikilik tabanda ekrana yazdırmak, örneğin `101011` gibi. Bunu özyinelemeli olarak nasıl yapabiliriz?

Bir öğrenci, sayıyı sürekli ikiye bölüp kalanı tutmayı önerdi: Her adımda ikiye böl, kalanı yaz. Hoca ikiye bölmenin $n/2$, kalanın da n mod 2 olduğunu gösterdi. Öğrenci kalanların ters sırada yazılması gerektiğini de hatırlattı.

Hoca burada ters çevirmeye gerek olmadığını vurguladı. `power(5, 3)` örneğinde, `power(5, 0)`'ın nereden çağrıldığını hatırlaması gerektiğini söylemişti. Benzer şekilde, sayı ikiye bölüne bölüne en derin çağrıya kadar inilir, sonra geri dönülür. Geri dönüş sırası, basamakları en anlamlıdan en anlamsıza doğru doğal olarak doğru sırada yazar. Bu yüzden ters düz etme işlemine ihtiyaç yoktur.

8.1 Temel Durumun Seçimi

Fonksiyonun adı `printInBinary` olsun ve bir n tam sayısı alsın. İki durum tanımlanmalıdır: temel durum ve özyinelemeli kısım. Bunun için bir `if` ile temel duruma varılıp varılmadığına karar verilir.

Hoca hangi kontrolün yapılacağını sordu. Bir öğrenci önce "$n/2 > 0$" önerdi, sonra vazgeçip "$n < 2$ mi diye bakalım" dedi. Hoca bunu doğruladı: $n < 2$ ise 2^1'den aşağıdayız, yani 2^0 basamağındayız, birler basamağına kadar inmişiz demektir. Bu son basamak 0 ya da 1'dir ve doğrudan yazdırılır. Bu temel durumdur; fonksiyon artık kendini çağırmaz.

Temel durumda ne yazılacağını sorduğunda bir öğrenci önce `return n` önerdi. Fonksiyon `void` olduğu için hoca bunun yerine `printf` ile n'nin yazdırılması gerektiğini söyledi: Bir ise bir, sıfır ise sıfır yazılır.

$n \geq 2$ ise henüz temel duruma varılmamıştır. Sayı ikiye bölünerek fonksiyon kendini çağırır; böylece basamaklar ilerletilir. Ama hoca bunun yeterli olmadığını söyledi: Bu çağrıdan sonra son basamağı (1 ya da 0) yazdırmak için fonksiyon kendini bir kez daha, n mod 2 ile çağırmalıdır.

```
void printInBinary(int n) {
    if (n < 2)
        printf("%d", n);          /* temel durum */
    else {
        printInBinary(n / 2);
        printInBinary(n % 2);
    }
}
```

8.2 $n = 43$ İçin İzleme

Hoca slayttaki bir ikilik gösterimi sayarak 43 sayısını seçti ($1 + 2 + 8 + 32 = 43$). İzlemede fonksiyon kısaca `f` diye anıldı.

İniş:

- `f(43)`: $43 < 2$ değil. `f(43 / 2)` = `f(21)` çağrılır.
- `f(21)`: $21 < 2$ değil. `f(10)` çağrılır.
- `f(10)`: $10 < 2$ değil. `f(5)` çağrılır.
- `f(5)`: $5 < 2$ değil. `f(2)` çağrılır.
- `f(2)`: $2 < 2$ değil. `f(1)` çağrılır.
- `f(1)`: $1 < 2$. Temel durum; ekrana **1** yazılır.

Çıkış (her seviyede ikinci çağrı `f(n % 2)` çalışır; bu değer hep 0 ya da 1 olduğu için doğrudan temel duruma düşer):

Dönülen çağrı	İkinci çağrı	Ekrana yazılan
`f(2)`	`f(2 % 2)` = `f(0)`	0
`f(5)`	`f(5 % 2)` = `f(1)`	1
`f(10)`	`f(10 % 2)` = `f(0)`	0
`f(21)`	`f(21 % 2)` = `f(1)`	1
`f(43)`	`f(43 % 2)` = `f(1)`	1

Ekranda `101011` görülür. Bu, 43'ün ikilik karşılığıdır (C programıyla doğrulandı).

Hoca kodun ne kadar kısa ve basit olduğunu vurguladı. Bilinen yolla yapılsaydı bir döngü kurmak, bir diziyi doldurmak ya da tersten ilerlemek gerekirdi; kod ve akış daha uzun olurdu. Burada ise tek bir `if`, temel durum ve iki farklı parametreyle arka arkaya iki çağrı yeterli oldu.

8.3 Öğrencinin Önerisi: İkinci Çağrı Yerine `printf`

Soru: İkinci kez kendimizi çağırmak gereksiz değil mi? İkinci çağrı yerine doğrudan n mod 2'yi yazdırırsak hem çağrı yığını, hem kontrol, hem işlem açısından kazançlı olmaz mı?

Öğrencinin önerisi, `else` içindeki ikinci satırı silip yerine `printf("%d", n % 2);` yazmaktı.

Cevap: Hoca öneriyi 11 sayısıyla denedi: `f(11)` → `f(5)` → `f(2)` → `f(1)` çağrıldı. `f(1)` için önce temel durumda 1 yazdırdı, ardından 1 mod 2'yi de yazdırarak fazladan bir 1 çıkacağını söyledi ve önerinin işe yaramadığı sonucuna vardı. Hocaya göre çıktı yalnız temel durumda üretilmelidir; başka yerde de çıktı üretilirse nereden gelindiğini kontrol etmek gibi karmaşık kontroller gerekir ve kodun "büyüsü" kaçar.

> **Ders sonrası not:** Hocanın bu izlemesi yanlıştır. `f(1)` temel durumdur ve `if-else` yapısında `else` kolu çalışmaz; yani `f(1)` içinde n mod 2 yazdırılmaz. Doğru izleme: `f(1)` "1" yazar, geri dönüşte `f(2)` "0", `f(5)` "1", `f(11)` "1" yazar. Sonuç `1011`'dir ve 11'in doğru ikilik karşılığıdır. Öğrencinin önerisi doğrudur ve her seviyede bir fonksiyon çağrısını kurtarır. İki sürüm de C ile derlenip 43 ve 11 için çalıştırıldı; ikisi de `101011` ve `1011` yazdı.

```c
void printInBinary(int n) {
    if (n < 2)
        printf("%d", n);
    else {
        printInBinary(n / 2);
        printf("%d", n % 2);
    }
}
```

8.4 İki Çağrının Bedeli

Hoca, fonksiyonun kendi içinde kendini iki kez çağırdığına dikkat çekti. Bunun hem bellek hem işlem sayısı açısından yük getirdiğini söyledi: Kod çok kısa ve güzel çalışıyor, ama algoritmik açıdan ek yük vardır. Ayrıca 11 içinde 5, 5 içinde 2, 2 içinde 1 çağrıldığı için "beni kim çağırmıştı" bilgisini tutmak da ek bellek ister.

> **Hocanın vurgusu:** Bu bir fiyat-performans (trade-off) analizidir. Mantıklıysa kullanın. Özyinelemeli yapıları kullanmak her zaman iyi demek değildir.

> **Ders sonrası not:** Bu fonksiyonda ikinci çağrı her zaman 0 ya da 1 ile yapılır ve hemen temel duruma düşer. Bu yüzden iki çağrı karmaşıklığı katlamaz: Yineleme denklemi $T(n) = T(n/2) + c$'dir ve fonksiyon $O(\log n)$ sürede çalışır.

9. Özyinelemeli Algoritmalarda Büyük O: Yineleme Denklemi

Hoca, özyinelemenin neden her zaman iyi olmadığını görmek için geçen hafta görülen algoritma analizine, büyük O gösterimine döndü. Soru şudur: Özyinelemeli bir algoritmanın asimptotik üst sınırı nasıl bulunur, yani girdi sayısı n'ye bağlı çalışma zamanı fonksiyonu $T(n)$ nasıl kurulur?

Normal algoritmalarda olduğu gibi özyinelemeli fonksiyonların zaman karmaşıklığı da $T(n)$ ile ifade edilir. Ama $T(n)$ kendi cinsinden yazılır. Bu ifadeye yineleme denklemi (recurrence equation) denir. Bir özyinelemeli algoritmanın büyüme oranını bulmak için önce bu yineleme ilişkisini kurmak, sonra çözmek gerekir.

Kurmak için iki durum kullanılır: temel durum ve özyineleme durumu. İki durumun maliyetleri yazılıp birbiri cinsinden ifade edilir.

9.1 Örnek 1: Tek Çağrılı `f`

```
void f(int n) {
    if (n > 0) {
        printf("%d", n);
        f(n - 1);
    }
}
```

Fonksiyon $n > 0$ olduğu sürece sabit bir işlem yapar (n'yi yazdırır) ve kendini bir azaltarak çağırır. Hoca üç şartı kontrol etti: Özyinelemeli mi? Evet. Problem küçülüyor mu? Evet. Temel durum var mı? Evet: $n = 0$ olduğunda fonksiyon hiçbir şey yapmaz, yani durur. Temel durum boştur.

`f(4)`'ün izlenmesi: $4 > 0$, ekrana 4 yazılır ve `f(3)` çağrılır. Sonra 3 yazılır, `f(2)`; 2 yazılır, `f(1)`; 1 yazılır, `f(0)`. $0 > 0$ değildir; hiçbir şey çalışmaz ve özyineleme kesilir. Ekranda `4 3 2 1` görülür.

Hoca, hiçbir şey yapmasa bile fonksiyonun temel durumda kesin bir iş yapmış sayıldığını söyledi. Temel duruma $n = 0$'da varılır. Bu yüzden:

- $T(0) = a$ (sabit).
- $T(n) = b + T(n-1)$: Sabit bir işlem ve bir azaltılmış boyutta kendine bir çağrı.

Toplandığında: `f(4)` için sabit + `f(3)`, sabit + `f(2)`, sabit + `f(1)`, sabit + `f(0)` yani a. Toplam, birkaç tane b artı a kadar işlemdir.

> **Ders sonrası not:** Hoca bu denklemin sonucunu söylemedi. İterasyonla $T(n) = n \cdot b + a$ bulunur, yani $O(n)$.

9.2 Örnek 2: İki Çağrılı `g`

```
int g(int n) {
    if (n == 1)
        return 2;
    else
        return 3 * g(n / 2) + g(n / 2) + 5;
}
```

Temel durum $n = 1$'dir ve 2 döndürülür. Değilse fonksiyon kendini $n/2$ elemanla iki kez çağırır: Birinci çağrının sonucu 3 ile çarpılır, ikincininki olduğu gibi alınır ve 5 eklenir.

- $T(1) = c$: `return 2` sabit bir iştir.
- $T(n) = b + 2T(n/2)$: 3 ile çarpmak, 5 eklemek ve döndürmek sabit sayıda adımdır (b). Asıl önemli olan, fonksiyonun kendini iki kez $n/2$ ile çağırmasıdır.

Hoca, özyinelemeli algoritmalarda zaman karmaşıklığı yazılırken her zaman önce temel durumun, sonra özyineleme durumunun yazılması gerektiğini; problemin ne kadar küçültülerek ve kaç kez çağrıldığının toplamda ifade edilmesi gerektiğini söyledi.

Soru: `else`'ten sonraki iki çağrı farklı olabilir mi? Örneğin biri $g(n/2)$, diğeri $g(n/3)$ olabilir mi, yoksa hepsi aynı mı olmak zorunda?

Cevap: Kesinlikle aynı olmak zorunda değildir. Buradaki örnek basit olsun diye iki tane $n/2$ ile çağrılmıştır. Fibonacci'de biri $n-1$, diğeri $n-2$ ile çağrılacaktır. Tamamen problemin çözümünün yapısına bağlıdır. Biri $n-1$, diğeri $n/2$ ile de çağrılabilir; buna göre toplam karmaşıklık değişir.

Soru: Bir durum oluşana kadar fonksiyonu çağırıyoruz. Bunun `while` döngüsüne göre avantajı ya da dezavantajı var mı?

Cevap: Hoca, `while` döngüsüyle kendini ikiye bölerek çağıramayacağını söyledi. `f(4)` içinde `f(3)`, onun içinde `f(2)` çağrılıyor ve elde edilen sonuç kendini çağırana geri döndürülüyordu. `while` döngüsünde bir önceki adıma değer döndürülemez; döngü yalnız ilerler. Özetle döngüyle birebir aynı işi yapamazsınız. Ama birçok problemi döngü kullanarak doğrusal bir çözüme

dönüştürebilirsiniz. Hatta karmaşıklığı düşürdüğü için bu tercih edilir. Hoca, Fibonacci'de çok basit ama karmaşıklığı felaket bir çözüm göreceklerini söyledi.

> **Ders sonrası not:** Her özyinelemeli algoritma, çağrı yığınının yaptığı işi açık bir yığın (stack) veri yapısıyla taklit ederek döngüyle de yazılabilir. Hocanın "birebir aynı işi yapamazsın" sözü, `while`'da doğrudan bir geri dönüş mekanizması olmadığını anlatır.

10. Sık Görülen Yineleme Denklemleri

Hoca slaytta sık karşılaşılan yineleme denklemlerini ve sonuçlarını gösterdi. Slaytta sonuçların Θ ile verildiğini, ama bunların büyük O olarak da okunabileceğini söyledi. Ayrıntılarının gelecek yıl daha çok görüleceğini ekledi.

Yineleme denklemi	Sonuç	Hocanın açıklaması
$T(n) = T(n-1) + n$	$O(n^2)$	Bir azaltarak kendini çağırırken ayrıca n'ye bağlı bir döngü kuruluyor.
$T(n) = T(n/2) + c$	$O(\log n)$	Her adımda girdi yarıya iniyor ve ayrıca sabit sayıda işlem (örneğin bir kontrol) yapılıyor.
$T(n) = T(n/2) + n$	$O(n)$	Her adımda yarıya bölünüyor ama geri kalan elemanlar üzerinde n'ye bağlı bir döngü kuruluyor.
$T(n) = 2T(n/2) + c$	$O(n)$	Girdi iki yarıya bölünüyor, iki yarım üzerinde çalışılıyor ve sabit ek iş yapılıyor; `g` örneği.
$T(n) = T(n-1)+T(n-2)+c$	—	Hoca bunun bu kadar basit olmayacağını söyledi; Bölüm 13'te çözüldü.

Bu denklemlerin hepsinde ya tek bir T terimi ya da aynı T teriminden iki tane vardır. $T(n-1) + T(n-2)$ gibi farklı terimler olduğunda işler karışır.

11. Özyinelemeli İkili Arama ve Yineleme Denklemi

Hoca bu analizin nasıl yapıldığını ilk derslerde görülen ikili arama (binary search) üzerinden gösterdi. İkili aramayı bu kez özyinelemeli bir fonksiyon olarak yazdı:

```
int binarySearch(int A[], int x, int low, int high) {
    int mid;
    if (low > high)
        return 0;                                   /* bulunamadı (false), c1 */
    mid = (low + high) / 2;                         /* c2 */
    if (x == A[mid])
        return 1;                                   /* bulundu (true), c3 */
    else if (x < A[mid])
        return binarySearch(A, x, low, mid - 1);    /* T(n/2) */
    else
        return binarySearch(A, x, mid + 1, high);   /* T(n/2) */
}
```

Fonksiyona bir A dizisi, aranan x değeri ve aramanın yapılacağı aralığın başlangıç ve bitiş indisleri (`low` ve `high`) verilir. İlk çağrıda aralık 1'den n'ye kadardır.

Hoca kodun her satırının maliyetini tek tek yazdı:

- **`low > high`:** `high` sola, `low` sağa geçmişse x bulunamamıştır. Temel durumdur ve `false` döndürülür. Bu kontrol sabit bir c_1 zamanı alır.
- **`mid = (low + high) / 2`:** Orta indis bulunur. n'ye bağlı değildir; sabit c_2 zamanı alır.
- **`x == A[mid]`:** Aranan değer dizinin bu gözündeki elemana eşitse `true` döndürülür. Bu da sabittir: c_3. Bu, bulunarak durma şartıdır.
- **`x < A[mid]`:** Aranan değer ortadakinden küçükse `low` ile `mid - 1` arasında, yani sol yarıda aranır. Aynı problem, ama boyutu $n/2$'ye düşmüştür: $T(n/2)$.
- **`x > A[mid]`:** Sağ yarıda, `mid + 1` ile `high` arasında aranır. Yine $n/2$ boyutlu bir problemdir: $T(n/2)$.

Hoca bu kolların birbirini dışladığını (mutually exclusive) vurguladı. `return true` yukarıda olduğu için, o çalışırsa zaten durulur. Ya sabit iş, ya sol yarı, ya sağ yarı çalışır. Bu yüzden en kötü durum için en çok zaman alan kol seçilir: sabit, $T(n/2)$ ve $T(n/2)$ arasında maksimum $T(n/2)$'dir. Sabitler toplanınca:

$$T(n) = c + T(n/2)$$

Bu, en kötü durumda ikili aramanın çalışma zamanıdır; bundan daha fazla çalışmayacağı garanti edilir.

Hoca, $T(n)$'yi $T(n/2)$ cinsinden tanımladığımızı hatırlattı. Aynı şekilde $T(n/2)$, $T(n/4)$ cinsinden; $T(n/4)$ da $T(n/8)$ cinsinden yazılabilir. Yani n yerine hep $n/2$ yazılabilir.

12. Yineleme Denklemlerini Çözme Yöntemleri ve İterasyon

$T(n) = c+T(n/2)$ bulunduktan sonra bu denklemi çözmek gerekir. İlk iş, kendini özyinelemeli çağıran yapıyı bir seri biçiminde açmak ve özyineleme içermeyen, toplam çağrı sayısından oluşan bir ifadeye dönüştürmektir. Algoritma analizinde bunun için dört yöntem vardır:

- **İterasyon yöntemi** (iteration method).
- **Yerine koyma yöntemi** (substitution method): Bir çözüm varsayıp yerine koyarak doğrulamak.
- **Özyineleme ağacı yöntemi** (recursion tree method): Ağaç veri yapısı henüz görülmedi ama bu derste basitçe üzerinden geçilecek.
- **Master yöntemi** (master method): Kuralları net tanımlanmış ve teoremleri ispatlanmış olduğu için, belirli şartlar sağlandığında "şöyleyse budur" biçiminde doğrudan uygulanan bir karar dallanmasıdır.

Hoca hepsini anlatmak istediğini, ama dersin amacının veri yapılarına geçmek olduğunu ve vakit olmadığını söyledi. Bu derste yalnız iterasyon yöntemiyle, çok karmaşık olmayan özyinelemeli algoritmaların çalışma zamanının nasıl bulunacağı gösterildi.

> **Ek bilgi (derste anlatılmadı):** Master yöntemi $T(n) = aT(n/b)+f(n)$ biçimindeki denklemler için kullanılır ($a \geq 1$, $b > 1$). $f(n)$, $n^{\log_b a}$ ile karşılaştırılır ve hangisi baskınsa sonucu o belirler; eşitse bir $\log n$ çarpanı eklenir. Örneğin ikili aramada $a = 1$, $b = 2$, $n^{\log_2 1} = 1$ ve $f(n) = c$ olduğu için sonuç $\Theta(\log n)$ olur. Yöntem büyük olasılıkla gelecek yılki algoritma analizi dersinde ayrıntılı işlenir.

12.1 İkili Aramanın İterasyonla Çözümü

Başlangıç denklemi $T(n) = c + T(n/2)$'dir. Hoca önce sezgiyi bir sayıyla verdi: 20 elemanlı bir dizide sabit iş artı 10 elemanlık bir problem vardır. O da başka bir sabit artı 5 elemanlık problem demektir; o da sabit artı 2, o da sabit artı 1. Yani dizinin boyutu bir elemana inene kadar bölmeye devam edilir.

Sonra denklemi açtı:

- $T(n) = c + T(n/2)$.
- $T(n/2) = c + T(n/4)$ olduğu için yerine konur: $T(n) = c + c + T(n/4)$.
- $T(n/4) = c + T(n/8)$ olduğu için: $T(n) = c + c + c + T(n/8)$.
- k adım sonra: $T(n) = k \cdot c + T(n/2^k)$.

Bu açılım $T(1)$'e varana kadar sürer. $T(1)$'de algoritma yalnız `return false` ya da `return true` yapar; yani c_1 ya da c_3 kadar sabit bir iştir.

Hep 2'ye bölerek gidildiği için $n = 2^k$ yazılır. Böylece $n/2^k = 1$ olur. İki tarafın 2 tabanında logaritması alınınca:

$$k = \log_2 n$$

k yerine $\log n$ yazılınca:

$$T(n) = c \log n + T(1) = c \log n + c'$$

Büyük O gösteriminde küçük terimler atılır. $\log n$'ye bağlı bir terim varken sabit terimin "esamesi okunmaz". Katsayı c de atılır. Sonuç:

$$T(n) = O(\log n)$$

Hoca yapılan işi şöyle özetledi: Temel duruma kadar iterasyon yapılır. $n/2, n/4, n/8, n/16, n/32, \ldots, n/2^k$ diye gidilir. $n = 2^k$ denip logaritma alınınca $k = \log n$ olur. k tane c, $\log n$ tane c demektir; üstüne sabit bir iş eklenir. Büyüme oranı $\log n$ mertebesindedir.

Soru: Adım sayısı açısından döngüyle aynı çözümü veriyor. Ama bellek açısından döngüye göre $\log n$ mertebesinde bir olumsuz yanı var sanki?

Cevap: Algoritmanın kendi çalışma prensibi açısından ek bir bellek gereksinimi yoktur. Ama fonksiyonun kendini tekrar tekrar, $\log n$ kez çağırması gerektiği için, evet, $\log n$ mertebesinde "hangi fonksiyondan gelmiştim" bilgisini hatırlama alanına ihtiyaç vardır. Bu bilgi fonksiyon çağrı yığını (function call stack) denen mekanizmada tutulur. Hoca, yığınların iki hafta sonra işleneceğini, bu yüzden şimdilik ayrıntıya girmeyeceğini söyledi.

13. Özyinelemeli Fibonacci

Hoca, her özyinelemeli algoritmanın ikili arama kadar "temiz, güzel" olmadığını, $\log n$ gibi düşük bir karmaşıklığa sahip olmadığını göstermek için meşhur Fibonacci sayılarına geçti.

13.1 Fibonacci Dizisi

Fibonacci dizisinde ilk değerler sabittir; sonraki her sayı kendinden önce gelen iki sayının toplamıdır. Hoca, kimilerinin diziyi Fibonacci 1 ve Fibonacci 2'den, kimilerinin de Fibonacci 0 ve Fibonacci 1'den başlattığını söyledi ve 0, 1'den başlamayı seçti. Dizi:

$$0,\ 1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13,\ 21,\ 34,\ 55,\ 89,\ \ldots$$

Hoca ilk on terimi (0'dan 9'a kadar indislerle) yazıp 55'e, ardından 89'a kadar devam etti. Diziyi yazarken bir "1"i atladı; bir öğrencinin uyarısıyla dizinin 0, 1, 1, 2 diye gittiğini kabul edip düzeltti.

Dizi, adını onu tanımlayan Fibonacci'den alır. Hoca, dizinin doğada ve yaşamda pek çok yerde karşımıza çıktığını söyledi: tavşanların üremesi, ağaç dallarının belli bir seviyedeki sayısının artış oranı gibi. Ayrıntısını merak edenlerin araştırmasını istedi.

13.2 Kod

n. Fibonacci sayısını hesaplayan özyinelemeli kod çok basittir:

```
int fibonacci(int n) {
    if (n <= 1)
        return n;                                   /* temel durum */
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Temel durum $n \leq 1$, yani n'nin 0 ya da 1 olmasıdır ve n döndürülür: 0'dan 0, 1'den 1 gelir. Diğer her sayı için kendinden bir önceki (`fibonacci(n - 1)`) ve iki önceki (`fibonacci(n - 2)`) sayının toplamı döndürülür. Hoca bunun "çok temiz, dümdüz" bir ifade olduğunu söyledi.

13.3 Yineleme Denklemi

- $n = 0$ için $T(0)$: `return n`, sabit c_1.
- $n = 1$ için $T(1)$: sabit c_2.
- $n > 1$ için: $T(n) = C + T(n-1) + T(n-2)$.

Hoca burada "eyvah eyvah" dedi: $T(n)$'nin içinde T cinsinden iki farklı bileşen vardır. Fonksiyon kendini bir eksiğiyle ve iki eksiğiyle, iki kez çağırmak zorundadır. Üstelik $n-1$'in içinde $n-2$ yeniden çağrılır; $n-2$ ile $n-1$ içinde $n-3$'ler ikişer kez çağrılır, belki $n-4$'ler de. Birinci seviyede iki, ikinci seviyede dört, üçüncü seviyede sekiz çağrı yapılır. Aslında aynı şey tekrar tekrar çağrılır.

13.4 Üst Sınır İçin Yaklaşım

Bu denklemi olduğu gibi iterasyonla çözmek zordur. Hoca bir kabul yaptı: n'nin yeterince büyük değerleri için bir adım fazlası ya da eksiği ek bir yük sayılmayabilir. Yani:

$$T(n-2) \approx T(n-1)$$

Hoca aslında $T(n-1)$'in daha büyük olduğunu bildiğimizi hatırlattı: Örneğin Fibonacci 9'u hesaplamak için 9, 8, 7, ..., 1 çağrılır; Fibonacci 8 için 8, 7, ..., 1. Yani $T(n-1) \geq T(n-2)$'dir. $T(n-2)$ yerine daha büyük olan $T(n-1)$'i koyduğumuz için sonuç bundan daha kötü olamaz; bu bir üst sınır yaklaşımıdır.

> **Hocanın vurgusu:** Bu bir varsayımdır, unutmayın. Denklem bu kabulle yaklaştırıldı.

13.5 İterasyon

Kabulle denklem $T(n) = 2T(n-1) + c$ olur. Hoca açılımı adım adım yaptı:

- $T(n) = 2T(n-1) + c$.
- $T(n-1) = 2T(n-2) + c$ yerine konur: $T(n) = 2\big(2T(n-2) + c\big) + c = 4T(n-2) + 3c$. Burada $2c$ parantezden, c dışarıdan gelir.
- $T(n-2) = 2T(n-3) + c$ yerine konur: $T(n) = 4\big(2T(n-3) + c\big) + 3c = 8T(n-3) + 7c$.
- k adım sonra: $T(n) = 2^k T(n-k) + (2^k - 1)c$. Sabitlerin katsayıları 1, 3, 7, ... yani $2^k - 1$ biçiminde ilerler.

Amaç $T(n-k)$'yi temel duruma, $T(0)$'a indirmektir. Bunun için $n - k = 0$, yani $k = n$ alınır:

$$T(n) = 2^n T(0) + (2^n - 1)c$$

Hoca $T(0)$ yerine 1 koydu (ara adım eklendi: bu, temel durum maliyetini $c_1 = 1$ birim saymak demektir):

$$T(n) = 2^n + 2^n c - c = 2^n(1 + c) - c$$

$-c$ sabittir, önemsenmez. $(1 + c)$ bir katsayıdır ve büyük O gösteriminin kuralları gereği atılır. Sonuç:

$$T(n) = O(2^n)$$

Bu sonuç $T(n-1) \approx T(n-2)$ yaklaşımıyla elde edilmiştir. Özyinelemeli Fibonacci algoritmasının çalışma süresi 2^n oranında büyür.

Açılımdaki ara ifadeler ($4T(n-2)+3c$, $8T(n-3)+7c$, genel $2^kT(n-k)+(2^k-1)c$) ve kapalı biçim $2^n(1+c)-c$, Python ile denklem doğrudan hesaplanarak doğrulandı.

Hoca sonucu şöyle yorumladı: Kendinizi bir eksiğinizle ve iki eksiğinizle iki kez çağırdığınız anda üstel (exponential) karmaşıklıkta bir çözüme sahip olursunuz. Kod iki satırdır, ama karmaşıklığı 2^n'dir. Aynı iş, önceki iki değerin toplamını tutan dümdüz bir döngüyle yapılabilirdi; tek döngü artı sabit işlem, yani $O(n)$.

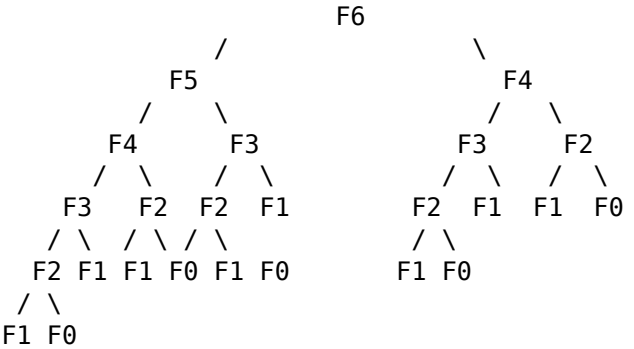
> **Hocanın vurgusu:** Çok basit ve masum görünen bir implementasyon, karmaşıklığın neredeyse kabul edilemeyecek oranlara çıkmasına sebep olabilir. Küçük değerler (10, belki 100) için mantıklı olabilir, ama büyük değerlerde ne kadar beklemeniz gerektiğini bu basit kodu yazarak kendiniz test edebilirsiniz.
>
> **Ders sonrası not:** $T(n)=2T(n-1)+c$ bir üst sınırdır. Gerçek denklem $T(n)=T(n-1)+T(n-2)+c$'nin çözümü $\Theta(\varphi^n)$'dir; burada $\varphi=(1+\sqrt{5})/2\approx 1{,}618$ altın orandır. Python ile gerçek denklemde $T(40)/T(39)\approx 1{,}618$ bulundu. Bu da üsteldir, yani hocanın "üstel karmaşıklık" sonucu değişmez; yalnız taban 2'den biraz küçüktür.

14. Özyineleme Ağacı: `fib(6)`

Hoca karmaşıklığın neden bu kadar büyüdüğünü görselleştirmek için `fib(6)`'nın çağrı ağacını çizdi.

- F6, F4 ($n-2$) ve F5'i ($n-1$) çağırır: Fibonacci 6, Fibonacci 4 artı Fibonacci 5 demektir.
- F4, F2 ve F3'ü çağırır.
- F2, F1 ve F0'ı çağırır. Hoca bunları sabit değerler olarak gösterdi.
- F3, F1 ve F2'yi çağırır; F2 de yine F1 ve F0'a dallanır.
- Öte yandan F5 de F3 ve F4'ü çağırır ve onlar da aynı biçimde dallanır.

Ağaç (sol dal $n-1$, sağ dal $n-2$ olarak çizildi; hoca tahtada F4'ü solda çizmişti):



Hoca aynı alt ağaçları renklerle işaretledi:

- F5'in altındaki F4 bloğu, F6'nın doğrudan çağırdığı F4 bloğuyla birebir aynıdır. Hoca bu bloktaki beş fonksiyon çağrısı ve üç sabit değer dönüşünün ikinci kez yapıldığını söyledi.
- F3 çağrısı ağaçta birden fazla yerde aynen tekrarlanır.
- F2 ve F1 de pek çok yerde yeniden çağrılır.

Bu aynı şeyleri tekrar tekrar hesaplamaktır. Hoca sayıları şöyle verdi: F2 dört kez, F3 üç kez, F4 iki kez, F5 bir kez çağrıldı.

> **Ders sonrası not:** Hoca F2'nin dört kez çağrıldığını söyledi; doğrusu beş kez. $n\le 1$ temel durumuyla `fib(6)` için çağrı sayıları şöyledir (C programında sayaçla doğrulandı): F6 1, F5 1, F4 2, F3 3, F2 5, F1 8, F0 5; toplam 25 çağrı. F3, F4 ve F5 için hocanın sayıları doğrudur.

Hoca ayrıca tek bir Fibonacci sayısı için toplam "14 fonksiyon artı 8 sabit değer çağrısı" yapıldığını saydı.

2^n'nin nereden geldiğini hoca ağacın seviyelenmesiyle açıkladı: Büyük F4 bloğu iki kez, F3 bloğu üç kez çalışmak zorundadır. Biz bunu biliyoruz ama kullanamıyoruz, çünkü körlemesine tekrar tekrar çağırıyoruz. Bu yüzden toplam iş 2^n mertebesine çıkar.

Hoca bunun, daha önce adını andığı üçüncü yöntemin, özyineleme ağacı yönteminin (recursion tree method) ta kendisi olduğunu söyledi: Fonksiyonun dallanmasını çizip ağacın ne kadar yüksek ve ne kadar geniş olduğuna göre karmaşıklığı hesaplamak.

Soru: Fibonacci 1'in değerinin 1 olması gerekmiyor mu?

Cevap: Hoca buna takılmamak gerektiğini söyledi. İlk iki değer 1, 1 ya da sıfırı da sayarsak ilk üç değer 0, 1, 1 alınabilir; bir yerden başlanır. Önemli olan ilk değerlerin sabit bilinmesi ve geri kalan her şey için önceki iki değerin toplanmasıdır. Bu toplama sırasında, örneğin Fibonacci 6 için Fibonacci 4 ve Fibonacci 5 çağrılmak zorundadır; her biri kendi içinde tekrar dallanır. Sonuçları bilsek bile her seferinde çağırıp döndürmek zorundayız, çünkü özyinelemeli çağrı bunu gerektirir.

Hoca bu algoritmanın özyineleme kurallarına uyduğunu tekrar kontrol etti: Temel durum var, problem küçülüyor, $n-1$ ve $n-2$ boyutlu daha küçük alt problemler çağrılıyor. Kurallara uygun, ama aynı problemi o kadar çok çağırıyor ki 2^n mertebesinde çalışma zamanı gerekiyor. İkili arama $\log n$ mertebesinde çalışır, çünkü sürekli ikiye bölerek gider ve diğer yarıyla hiç ilgilenmez. Fibonacci ise bir eksiğini ve iki eksiğini sürekli çağırmak zorunda olduğu için gereksiz çağrılardan dolayı çok uzun sürer.

> **Hocanın vurgusu:** Özyinelemeli yapılar her zaman iyi çözümler getirecek diye bir şey söz konusu değildir.

15. Daha İyi Çözüm: Döngüyle Fibonacci

Hoca en temel ve basit çözümü şöyle anlattı: Bir dizi kullanılır. Fibonacci 0 ve Fibonacci 1 bilinir. Bunlardan Fibonacci 2, ondan Fibonacci 3 ve sonraki değerler hesaplanır. Dizinin $i-2$ ve $i-1$ numaralı gözleri toplanıp i. göze yazılır; 3, 5, 8 diye yaza yaza ilerlenir.

- Bu çözüm tek bir doğrusal döngüyle çalışır: $O(n)$ zaman.
- n boyutunda bir dizi gerektirir: $O(n)$ bellek.
- Hepsini dizide tutmak zorunlu değildir. Sonuçlar yalnız ekrana yazılıyor ya da bir yere gönderiliyorsa, önceki iki değer ve şimdiki değer için sabit üç alan yeter; yani sabit bellekle de çözülebilir.

  > **Ek bilgi (derste anlatılmadı):** Özyinelemeli yapıyı koruyup tekrarları önlemenin bir yolu, hesaplanan her $F(i)$ değerini bir dizide saklamak ve aynı değer yeniden istendiğinde hesaplamadan döndürmektir. Buna hafızalama (memoization) denir ve hocanın gelecek yıl anılacağını söylediği dinamik programlamanın (dynamic programming) temel fikirlerinden biridir. Bu yolla her $F(i)$ bir kez hesaplanır ve özyinelemeli Fibonacci de $O(n)$ süreye iner.

15.1 Dersin Sonundaki Soru

Soru: (Bir öğrencinin yazılı sorusu) Önbellek (cache memory) de doluyor sanırım?

Cevap: Hoca hangi önbelleğin kastedildiğini sordu ve büyük olasılıkla fonksiyon çağrı yığınının kastedildiğini söyledi. Çok fazla dallanma olduğu için "nereden gelmiştim, beni kim çağırmıştı" bilgisinin tutulması gerekir ve bu çok fazla veri demektir. Bu, algoritmanın kendi tuttuğu iç veri değildir; algoritmanın çalışabilmesi için derleyicinin tutması gereken veridir. Algoritmanın çalışması için ihtiyaç duyduğu bellek alanı olmasa da, çalıştırılabilmesi için gereken bir bellek alanını mutlaka kullanır. Hoca bu durumda genellikle bir stack overflow oluşacağını söyledi.

> **Ders sonrası not:** Özyinelemeli Fibonacci'de çağrı yığınının derinliği aynı anda en fazla n kadardır, çünkü yığında yalnız o an etkin olan çağrı zinciri ($F(n), F(n-1), \ldots$) tutulur. Yani asıl sorun bellek değil, 2^n mertebesindeki toplam çağrı sayısıdır; stack overflow ancak n çok büyük olduğunda ortaya çıkar. "Cache memory" ifadesi işlemci önbelleğini anlatır ve buradaki konuyla büyük olasılıkla ilgisizdir.

Örnek Sorular

Soru 1 (derste sözlü, 00:12:39)

Aşağıdaki kod özyinelemeli bir algoritmanın gerçeklemesi midir?

```
int main() {
    main();
    return 0;
}
```

Çözüm (derste verildi): Hayır. Temel durum yoktur ve her çağrı aynı problemi küçültmeden tekrarlar; `main` kendini sonsuza kadar çağırır. Özyinelemenin ikinci ve üçüncü şartı sağlanmaz.

Soru 2 (derste sözlü, 00:30:58)

Bölüm 6'daki `mystery` fonksiyonu `mystery(648)` ile çağrılırsa hangi değeri döndürür?

Çözüm (derste verildi): `mystery(648)` → $a = 64$, $b = 8$ → `mystery(72)` → $a = 7$, $b = 2$ → `mystery(9)`. $9 < 10$ olduğu için 9 döner ve değişmeden en başa taşınır. Sonuç **9**'dur (C ile doğrulandı).

Soru 3 (derste sözlü, 00:37:32 ve 00:45:23)

5 diskli Hanoi Kuleleri problemini özyinelemeli olarak nasıl çözersiniz? Algoritma hangi parametreleri tutmalıdır?

Çözüm (derste verilen kısım): n diskli problem için önce üstteki $n-1$ disk yedek çubuğa taşınır, en büyük disk hedefe taşınır, sonra $n-1$ diskli problem aynı yolla çözülür. Tek diskli problem temel durumdur. Algoritma taşınan diski, kaynak, hedef ve yardımcı çubukları parametre olarak tutmalıdır.

> **Çözüm (derste verilmedi):** Standart özyinelemeli çözüm aşağıdadır. `gcc -Wall` ile derlendi; 5 disk için 31 hamle yazdırır. Hamle sayısı $T(n) = 2T(n-1)+1$, $T(1) = 1$ denkleminden $2^n - 1$'dir; yani algoritma $O(2^n)$'dir.

```
void hanoi(int n, char kaynak, char hedef, char yardimci) {
    if (n == 1) {
        printf("Disk 1: %c -> %c\n", kaynak, hedef);
        return;
    }
    hanoi(n - 1, kaynak, yardimci, hedef);
    printf("Disk %d: %c -> %c\n", n, kaynak, hedef);
    hanoi(n - 1, yardimci, hedef, kaynak);
}

/* hanoi(5, 'A', 'C', 'B'); */
```

Soru 4 (derste sözlü, 01:01:42 ve 01:05:47)

Onluk tabanda verilen bir n sayısını ikilik tabanda yazdıran özyinelemeli bir fonksiyon yazın. Temel duruma varıldığını hangi kontrolle anlarsınız? Fonksiyon 43 için ne yazdırır?

Çözüm (derste verildi): Kontrol $n < 2$'dir; bu durumda n (0 ya da 1) doğrudan yazdırılır. Aksi halde önce `printInBinary(n / 2)`, sonra `printInBinary(n % 2)` çağrılır (Bölüm 8.1). 43 için çağrılar `f(43)`, `f(21)`, `f(10)`, `f(5)`, `f(2)`, `f(1)` biçiminde iner; geri dönüşte ekrana `101011` yazılır (C ile doğrulandı).

Soru 5 (derste sözlü, 01:15:03)

Soru 4'teki fonksiyonda ikinci özyinelemeli çağrı `printInBinary(n % 2)` yerine `printf("%d", n % 2)` yazılırsa çıktı doğru olur mu? 11 için deneyin.

Çözüm: Derste hoca fazladan bir 1 yazılacağını söyledi; bu yanlıştır (bkz. Bölüm 8.3'teki ders sonrası not). `f(1)` temel durumdur ve `else` kolu çalışmaz. Çıktı `1011` olur ve doğrudur (C ile doğrulandı). Değişiklik her seviyede bir fonksiyon çağrısı kazandırır.

Soru 6 (derste sözlü, 01:27:32)

Özyinelemeli bir fonksiyondaki iki çağrı farklı parametrelerle yapılabilir mi, örneğin biri $n/2$, diğeri $n/3$ ile?

Çözüm (derste verildi): Evet. Çağrıların aynı olması gerekmez; problemin yapısına bağlıdır. Fibonacci $n-1$ ve $n-2$ ile çağırır. Çağrılar değişince yineleme denklemi ve karmaşıklık da değişir.

Soru 7 (derste sözlü, 01:45:29)

Özyinelemeli ikili arama, döngüyle yazılan ikili aramaya göre ek bellek harcar mı?

Çözüm (derste verildi): Algoritmanın kendi verisi açısından hayır. Ama fonksiyon kendini $\log n$ kez iç içe çağırdığı için çağrı yığınında $O(\log n)$ ek yer tutulur.

Dersin Toparlaması

Bu derste özyineleme, bir problemi aynı türden daha küçük alt problemlere indirip aynı çözümü küçük örneklere uygulama yaklaşımı olarak tanımlandı. Doğru bir özyinelemeli yapı için üç şart gerekir: Bütün geçerli girdiler karşılanmalı, özyineleme içermeyen bir temel durum olmalı ve her çağrı temel duruma doğru küçülmelidir. Kendini koşulsuz çağıran `main` bu şartları sağlamaz. Üs alma, `mystery`, Hanoi Kuleleri ve ikilik tabanda yazdırma örnekleri, çağrıların yukarıdan aşağı yapıldığını ve sonuçların aşağıdan yukarı toplandığını gösterdi. Özyineleme kodu kısaltır ve akışı sadeleştirir; ama çağrı bilgisinin tutulması ek bellek ister ve karmaşıklık çoğu zaman artar.

İkinci yarıda özyinelemeli algoritmaların analizi için yineleme denklemi kuruldu ve iterasyon yöntemiyle çözüldü. İkili aramanın denklemi $T(n) = c + T(n/2)$'den $O(\log n)$, özyinelemeli Fibonacci'nin $T(n) = c+T(n-1)+T(n-2)$ denkleminden, $T(n-1) \approx T(n-2)$ yaklaşımıyla $O(2^n)$ bulundu. `fib(6)` ağacı aynı alt problemlerin tekrar tekrar hesaplandığını gösterdi; aynı iş bir döngüyle $O(n)$ sürede yapılabilir. Hoca özyinelemenin her zaman iyi bir çözüm olmadığını, bir fiyat-performans analizi yapılması gerektiğini vurguladı. Bir sonraki derste bağlı listelere (linked list) geçilecek. Hocanın belirttiğine göre fonksiyon çağrı yığınının ayrıntısı da iki hafta sonra yığınlar (stack) konusunda görülecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.